

Running uvicorn server

- `python -m uvicorn app.main:app --host 127.0.0.1 --port 8002 --workers 14`

Rebuild Containers with Forced Recreate

- `docker compose -f docker-compose.prod.yml up --build --force-recreate -d`

Force Docker to pick up and apply the fresh FastAPI endpoint configuration

- `docker compose -f docker-compose.prod.yml up -d --force-recreate backend-api`

Rebuild Containers with Clean Cache for frontend

- `docker compose -f docker-compose.prod.yml build --no-cache frontend-next && docker compose -f docker-compose.prod.yml up -d`

Access production_frontend shell

- `docker exec -u 0 -it production_frontend sh`

Verify Network Services Boot Successfully

- `docker compose -f docker-compose.prod.yml ps`

Tail Live Logs from the Backend Monitoring Portal

- `docker logs -f production_backend`

Tear Down the System Completely

- `docker compose -f docker-compose.prod.yml down`

Run Development Server on selected port

- `npm run dev --port 3001`

Run the Task Immediately – Execute this command in your host terminal to point Python directly to that file and run the function:

- `docker exec -it production_backend python -c "from tasks.data_automation import automate_monthly_report; automate_monthly_report()"`

What to Do if You Get a "ModuleNotFoundError"

- If Python still complains that it can't find tasks, it means it doesn't recognize /app as the root package directory.
- You can fix this instantly by passing the PYTHONPATH variable directly into the command like this:

- `docker exec -it -e PYTHONPATH=/app production_backend python -c "from tasks.data_automation import automate_monthly_report; automate_monthly_report()"`
- `docker exec -it -e PYTHONPATH=/app production_backend python -m tasks.data_automation`

Verify the Data Automation Outputs

- `docker exec -it production_backend cat cache/latest_report.json`

Run the `automate_monthly_report` function

- `docker exec -it production_backend python -c "from tasks.data_automation import automate_monthly_report; automate_monthly_report()"`

Force Docker to pick up and apply the fresh FastAPI endpoint configuration

- `docker compose -f docker-compose.prod.yml up -d --force-recreate backend-api`

Stop all services and completely destroy attached volume caches

- `docker compose -f docker-compose.prod.yml down --volumes --remove-orphans`

Remove all cached and downloaded Docker images

- `docker rmi $(docker images -q) -f`

Force-delete any leftover Docker volumes matching your project

- `docker volume rm $(docker volume ls -q | grep postgres_production_data) 2>/dev/null || true`

Relaunch the infrastructure network freshly

- `docker compose -f docker-compose.prod.yml up -d`

TESTS

- Backend Pytests
 - `docker exec -it -e PYTHONPATH=/app production_backend python -m p`

Postgresql

- `sudo -u postgres psql`

Login psql shell without sudo

- `psql -U postgres -h localhost`
- **list users:**
 - `\du`
 - `psql -U postgres -h localhost -c "\du"`
- **list databases**
 - `psql -U postgres -h localhost -l`

postgresql master_invoices table

- **show table:**
 - `docker exec -it production_postgres psql -U production_user -d production_db -c "\d master_invoices"`
- **show content:**
 - `docker exec -it production_postgres psql -U production_user -d production_db -c "SELECT id, user_id, signup_date, billing, source_file, email_message_id FROM master_invoices;"`
- **drop table:**
 - `docker exec -it production_postgres psql -U production_user -d production_db -c "DROP TABLE master_invoices;"`

404 Not Found error for newly added routes in FastAPI inside a Docker container almost always means

- Uvicorn is serving an old, cached layer of your code instead of hot-reloading your changes.
- If you modify backend/app/main.py on your host computer, those changes do not automatically sync inside the running container unless you have configured a Docker development volume mount or explicitly forced a cache rebuild.
- Follow this rapid diagnostic checklist to fix the route discovery blockage immediately:
- **The Direct Fix:**
 - Force a Hard Cache Rebuild To bypass Docker's internal layer caching and force Uvicorn to read your fresh route mappings,
 - run this command block in your terminal to take down the active container instance safely:
 - `docker compose -f docker-compose.prod.yml down`
- Re-compile the image strictly ignoring cached filesystem layers
 - `docker compose -f docker-compose.prod.yml build --no-cache backend-api`
- Spin the network cluster back up in background daemon mode
 - `docker compose -f docker-compose.prod.yml up -d`

Python Virtual Env

- `python3 -m venv venv`
- `source venv/bin/activate`

Force pytest to run explicitly from the current environment's Python path:

- `python -m pytest`
- `python -m pytest --cov=app`
- `pytest --cov=app tests/`

GIT

How to Reset Your Commits

1. Open the integrated terminal in VS Code (Ctrl + ~ on Windows/Linux or Cmd + ~ on Mac).
2. Run the command to go back two commits while keeping your files safe:
 - `git reset --soft HEAD~2`

GEMINI API KEY

- **API KEY:**
 - `AQ.Ab8RN6KB2vPu89blg-st9ZEY80MkOziGzYahdFc1HYXD8raqOA`
- **Name:**
 - Gemini API Key
- **Project name:**
 - `projects/161399373241`
- **Project Number:**
 - 161399373241

Authenticate Google Cloud Gmail API

1. Run script:
 - `docker exec -e PYTHONPATH=/app production_backend sh -c 'python -m tasks.gmail_script'`
2. Click on URL output in console
3. Go through authentication steps in the browser.
4. Copy url of final blank page
5. Open backend_api shell and curl to url
6. gmail_script will finish and future runs will already be authenticated

Add new self-hosted runner · Rsypertjr/auto-workflows2

- **Create a folder**
 - `mkdir actions-runner && cd actions-runner`
- **Download the latest runner package**
 - `$ curl -o actions-runner-linux-x64-2.336.0.tar.gz -L https://github.com/actions/runner/releases/download/v2.336.0/actions-runner-linux-x64-2.336.0.tar.gz`
- **Optional: Validate the hash**

- `echo "04cf0be1aff4c3ec3554466c39124ca250e3effd8873bb7e8d68535aa9505d5d actions-runner-linux-x64-2.336.0.tar.gz" | shasum -a 256 -c# Extract the installer`
- `tar xzf ./actions-runner-linux-x64-2.336.0.tar.gz`
- **Configure - Create the runner and start the configuration experience**
 - `./config.sh --url https://github.com/Rsyptertjr/auto-workflows2 --token AAXWOD6HI3CVWHDCUCMX3DLKMOFBI#`
- **Last step, run it!**
 - `./run.sh`

Using your self-hosted runner

- **The Production Fix:**
 - Set It to Run as a Permanent Background Service
 - Install the script into your system background services
 - `sudo ./svc.sh install`
 - Launch the permanent background daemon process
 - `sudo ./svc.sh start`
 - Verify that the automation layer is successfully listening
 - `sudo ./svc.sh status`